

CMPINF-2100 · COHORT NOTE SHARE

EDA Project Canyon Notes

The concepts behind an exploratory data analysis,
woven from the whole semester and run on a dataset you already know by heart.

Modules 1 · 4 · 6 · 7 · 8 · 9 · 10, applied



Codeman says: explore before you model.

Double Diamond QC · every number executed, twice-audited, gate-checked

Victor S. (DatJavaClass) · 2026-07-17

How to read this note

One connected lesson, not eight separate ones. Read it front to back once, then keep it for the parts you need.

A Cliff note is terse. A Cliff Jumper note weaves one module into a single lesson. A Canyon Note weaves the entire half-semester or project into one connected, illustrated lesson. This one covers the project phase: it walks the full arc of an exploratory data analysis (an EDA) and, at every step, points back to the module where the semester taught us that move.

KEY IDEA

The load-bearing fact of a section. If you remember one thing per part, make it this one.

→ THROUGH-LINE

A thread that keeps resurfacing across modules. The weave callouts are the soul of a Canyon Note.

WORKED EXAMPLE / TRACE

Real code, run for real, with its real output. Every number in this note was executed, not recalled.

⚠ TRAP

A place where a reasonable person gets it wrong. Read these twice.

COURSE'S-WAY NOTE

Where you should answer the way the course teaches it, even if the wider world has more to say.

What this note is (and is not)

This note teaches the *concepts* an EDA is made of, using only what our modules taught. It is not a walkthrough of any graded assignment, it contains no assignment questions and no submitted answers, and it will not do your project for you. It shows you the ideas so that your own analysis, on your own dataset, is yours. That is the deal, and it is a good one.

The dataset: the Note Share, exploring itself

Every worked example runs on `noteshare_snapshot_2026-07-17.csv`, which ships alongside this PDF (the canonical copy lives in the `CMPIFCohortNoteShare` repo). It is a snapshot of the Note

Share itself, taken 2026-07-17: **one row per shared file**. The Cliff notes you skim, the Cliff Jumper notes you study, the tools, the guides, this very document's older sibling: 87 rows, 11 columns. You already know these observations personally, which makes every plot self-checkable. When a bar looks tall, you can ask yourself whether the repo you know agrees.

Column	What it means	Type in storage
<code>file_path</code> , <code>file_name</code>	which file this row is	text
<code>section</code>	top-level grouping: the ten "Cliff Notes Module NN" folders collapse into one <code>Cliff Notes</code> value; other folders keep their names; loose files are <code>(root)</code>	text
<code>module</code>	module number parsed from the path; blank where the path carries no module number	number (watch this one)
<code>file_format</code>	extension, or <code>submodule</code> for linked companion repos	text
<code>size_kb</code> , <code>word_count</code> , <code>line_count</code>	how big; word and line counts exist for text files (md, txt) only	numbers
<code>n_commits</code>	how many commits ever touched the file: our stand-in outcome	number
<code>days_shared</code> , <code>days_since_update</code>	age since first commit; staleness since last commit	numbers

One honesty note about the figures: they are styled to match this document. Run the same code yourself and you will get seaborn's default look: same bars, same boxes, same numbers, different colors and, in places, a different bar order. Trace outputs are trimmed for width and shown to two decimals. The content is identical; only the paint differs.

The through-lines

- **The table.** Rows are observations, columns are attributes. Module 1 planted it; every part below stands on it.
- **Marginal, then joint, then conditional.** One variable, then two, then two split by a group. The seeing-order of an EDA.
- **The plot chooser.** Data type + unique values + one-or-relationship decides the plot. Every time.
- **Split-apply-combine.** One engine behind `value_counts` , `groupby` , hue, and facets.
- **NaN as connective tissue.** Missingness found early shapes every later statistic.
- **ddof=1.** Module 4's $n-1$ divisor, quietly inside every pandas `std()` .
- **The outcome in view.** Exploration feeds modeling; we keep one eye on the variable we would later predict.
- **The meta hook.** The dataset is the Note Share. You are inside the data.

What an EDA is

Before a single plot: where exploring sits in the pipeline, and why it exists.

Module 1 gave us the course's data science pipeline, based on the CRISP-DM framework. Six steps, and you will walk all of them this term:

Question **1**
research question or
business objective

Select data **2**
find and understand
datasets

Prepare & clean **3**
the famous "80%"

Explore **4**
distributions,
counts, summary
statistics: *you are here*

Model **5**
statistical patterns, prediction

Communicate **6**
tell the story to stakeholders

The Module 1 pipeline. Exploration is step 4, and it exists to feed step 5.

KEY IDEA

Exploring data means identifying and visualizing basic trends: distributions, counts of values across attributes, and summary statistics. It is the step where you find out what your data actually looks like before you ask a model to find patterns in it.

⚠ TRAP

The famous "80%" belongs to step 3: preparing and cleaning is "often called kind of the 80% of what we're doing." Exploration is its own step. But the lecture is careful to say the two loop: you clean, you explore, an outlier pops up, and you go back to cleaning to decide whether it is a mistake to remove or something important. EDA lives inside that loop; it is not the 80% itself.

Step 5 is where the pipeline is headed. In predictive modeling we relate other attributes to one attribute we care about, called the **outcome**. Module 1's examples: developing diabetes, or losing a customer. Exploration serves that future step by showing which attributes look associated with the outcome before any model is built. For our worked dataset we nominate `n_commits`, the revision count, as the outcome to keep in view: what kind of shared file keeps getting revised?

→ THROUGH-LINE

The table. Module 1 defined data as a table of rows (observations) and columns (attributes). NumPy gave the table axes, pandas gave it names, and every plot below is a picture of some slice of it.

Meet the table

Load it, look at it, measure it. Three moves, thirty seconds, and you know what you are holding.

pandas reads a CSV into a DataFrame with one call, from a local file or straight from a URL (Module 6 read a sample CSV both ways; Module 8 pulled GapMinder off GitHub by URL). Our file sits next to this PDF:

```
import pandas as pd

df = pd.read_csv("noteshare_snapshot_2026-07-17.csv")
df.head()  # first 5 rows, the smell test
```

TRACE · EXECUTED ON THE ANCHOR STACK

	file_name	section	module	file_format	size_kb	word_count
0	.gitattributes	(root)	NaN	none	0.0	NaN
1	.gitignore	(root)	NaN	none	0.4	NaN
2	.gitmodules	(root)	NaN	none	0.9	NaN
3	CMPIF2100_Part1_Canyon_Notes.pdf	Canyon Notes	NaN	pdf	3243.4	NaN
4	README.md	Canyon Notes	NaN	md	3.4	515.0

(6 of the 11 columns shown for width.)

Five rows in and the data is already talking: there are NaN s in `module` and `word_count` , and row 3 is enormous. Hold those thoughts; Parts III and V are about them. Now the two measuring moves:

```
df.shape  # (rows, columns): an attribute, so no parentheses
df.dtypes # what pandas thinks each column is
```

TRACE

```
(87, 11)

file_path      object
file_name      object
section        object
module         float64
file_format    object
size_kb        float64
word_count     float64
line_count     float64
n_commits      int64
days_shared   int64
days_since_update int64
dtype: object
```

87 observations, 11 attributes. Four columns are `object` , four are `float64` , three are `int64` .

COURSE'S-WAY NOTE

Quiz-question convention from Module 8, verbatim spirit: if a column's dtype says `object` , it is a string, and for this course that means a **categorical** variable. Answer it that way. (On our stack, pandas 2.3.3, text really does show as `object` .)

KEY IDEA

`shape` and `dtypes` are attributes (no parentheses); `head()` , `describe()` , `value_counts()` are methods (parentheses). pandas has been making you practice that distinction since Module 5. It still counts.

Know your variables

The types-and-uniques triage: which columns are categories, which are measurements, which are just names, and where the holes are.

Module 8's plot-choice lecture gave us the course's two working types. **Categorical** variables hold non-numeric values, especially strings. **Continuous** (also called numeric) variables hold integers and floats. Which plot you reach for later is determined mostly by the data type, the number of unique values, and whether you are looking at one variable or a relationship. So the triage starts with two counts we can run on everything:

```
df.isna().sum()    # True/False per cell, then True counts as 1 per column
df.nunique()       # unique values per column
```

TRACE

	isna().sum()		nunique()
file_path	0	file_path	87
file_name	0	file_name	82
section	0	section	7
module	19	module	10
file_format	0	file_format	6
size_kb	6	size_kb	60
word_count	16	word_count	71
line_count	16	line_count	58
n_commits	0	n_commits	8
days_shared	0	days_shared	12
days_since_update	0	days_since_update	7

(Two runs shown side by side for reading.)

Read the missingness first, because it has reasons. The 16 missing word and line counts are the files that are not text (5 PDFs, the notebook, 4 extensionless files like `LICENSE`, and 6 submodules). The 6 missing sizes are the submodules: they are pointers to other repos, not files with bytes. And the 19 missing `module` values sit exactly on the rows whose paths carry no module number: the 6 root files, 5 Guides, 4 Tools, both Canyon Notes files, plus the Cliff Jumper README and the Trainers README. Missingness in this dataset is not damage; it is structure.

The `module` column, a detective story

Look at `module` in the dtypes trace: `float64`. A module number! With 10 unique values! Stored as a float! Two lessons hide here:

KEY IDEA

dtype tells you storage, not role. `module` is numeric in storage but categorical in meaning: `module 7` is not "3.5 times `module 2`." Module 8's rule: a continuous-looking variable with few unique values (roughly 20 or fewer; the lecture's example was `year` with 12) can be treated as a category, each value its own bucket. With 10 values, `module` qualifies instantly.

⚠ TRAP

Why float and not int? Because an integer column that contains `NaN` becomes `float64`. Executed proof: `pd.Series([1, 2])` is `int64`, but `pd.Series([1, 2, None])` is `float64`. The 19 blanks force the whole column to float. When a "whole number" column surprises you with a decimal dtype, go count its NaNs.

Now apply the same fewer-than-20 rule honestly to the rest of the table and it bites back:

`n_commits` has 8 unique values, `days_since_update` has 7, `days_shared` has 12. The rule says they *can* be treated as categorical. The lecture also says it is the analyst's judgment call. Here the call goes the other way: these are counts and day-spans whose magnitudes mean something (10 commits really is twice 5), and `n_commits` is the outcome we want to model later, so we keep all three continuous. The rule is a prompt to think, not a law.

Labels are not variables

`file_path` is unique on all 87 rows, and `file_name` on 82 (only `README.md` repeats: six sections keep one). A text column that is unique or nearly unique per row works as a label for each observation, not as a variable with a distribution: a count plot of it has one bar per row, a `groupby` of it has one group per row, and no summary emerges. Set the two of them aside as identifiers and explore the other nine.

→ THROUGH-LINE

NaN as connective tissue. pandas has default answers for missing values everywhere: statistics skip them quietly, and Module 7 showed `groupby` dropping them from group keys. We found ours here; Parts IV and V will show those defaults acting on exactly these 19, 16, and 6.

Summarize before you draw

Counts for categories, moments for measurements, and the groupby engine that splits both by anything.

For a categorical variable, the summary is the count of each value. Module 8 phrased it as "the number of rows for each unique value," and `value_counts()` delivers it sorted most-frequent first by default:

```
df["section"].value_counts()
```

TRACE

section	
Cliff Notes	54
Cliff Jumper Notes	12
(root)	6
Guides	5
Tools	4
Trainers	4
Canyon Notes	2

For a continuous variable, the course-native move is `describe()`: count, mean, standard deviation, minimum, quartiles, maximum, in one call. Module 7 ran it on Titanic survival; we run it on `word_count`:

```
df["word_count"].describe()
```

TRACE

count	71.00
mean	906.93
std	686.79
min	232.00
25%	430.00
50%	725.00
75%	1099.50
max	3188.00

Notice `count 71.00`, not 87: the missing 16 sat out, silently. That is pandas's default NaN policy for statistics, and it is why Part III made you find the holes first: you cannot read "mean 906.9" honestly unless you know 16 rows had no say. When you want a chosen set of statistics instead of `describe`'s fixed menu, `.agg()` takes a list (the Module 7 notebook demonstrates `['mean', 'std']`):

```
df[["word_count", "n_commits"]].agg(["mean", "std"])
```

TRACE

	word_count	n_commits
mean	906.93	3.14
std	686.79	2.45

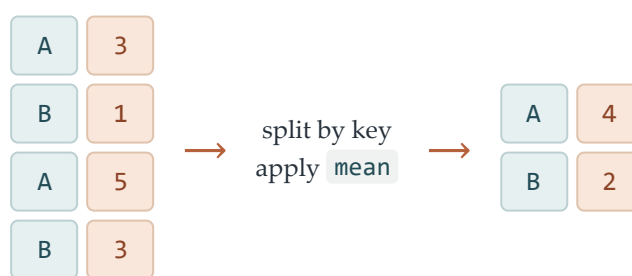
⚠ TRAP

Whose standard deviation? pandas `std()` divides by $n-1$ (the sample statistic, `ddof=1`, Module 4's divisor). NumPy's plain `np.std()` divides by n (`ddof=0`). Same column, two answers, both "correct": `df.word_count.std()` $\rightarrow$ 686.79, but `np.std(values)` $\rightarrow$ 681.94, and only `np.std(values, ddof=1)` $\rightarrow$ 686.79 agrees with pandas. If your numbers differ from a classmate's by a whisker, check the divisor before you panic.

The mean is the average (sum over how many, Module 4). The median is the middle point: half the data above, half below (Module 8 defined it while teaching boxplots). They tell different stories when the data is lopsided: `size_kb` has mean 64.1 KB but median 5.1 KB, because one 3.2 MB PDF drags the mean while the median shrugs. A right-skewed distribution typically pulls its mean above its median. Keep both in your pocket.

The engine: split, apply, combine

Module 7's centerpiece. `groupby` splits the rows into groups that share a value (usually of a categorical column), applies a function to each group, and combines the results into a new, smaller table: one row per group, a different shape than the input.



Split-apply-combine: 4 rows in, one row per group out.

```
df.groupby("section")["n_commits"].mean()
```

TRACE · SORTED FOR READING

section	
(root)	5.50
Cliff Jumper Notes	3.33
Cliff Notes	3.30
Trainers	2.25
Canyon Notes	2.00
Guides	1.00
Tools	1.00

Our outcome gets its first real signal: root-level files average 5.5 revisions, double most sections. Before you conclude "the cohort revises loose notes hardest," look at *which* files those are: the top three by `n_commits` are the repo's front README (19), the `.gitignore` (10), and the Cliff Jumper README (10). The most-revised files are the repo's front door and plumbing, not study notes. The front door gets the most paint. An EDA finding is only as good as your understanding of the observations behind it, and you happen to know these personally.

⚠ TRAP

`groupby` drops NaN group keys by default. Group our table by `module` and you get 10 groups; the 19 no-module rows vanish without a word. Module 7's Titanic lecture showed the fix when you care about the unlabeled rows: `df.groupby("module", dropna=False)` → 11 groups, the eleventh being the NaN crowd. Executed here: 10 vs 11. Silence is not absence.

→ THROUGH-LINE

`ddof=1`. Module 4 derived the $n-1$ sample variance by hand; Module 5 passed `ddof=1` to NumPy; today it is the silent default inside every pandas `std()` you run. Same idea, three costumes, zero new formulas.

Marginal distributions: one variable at a time

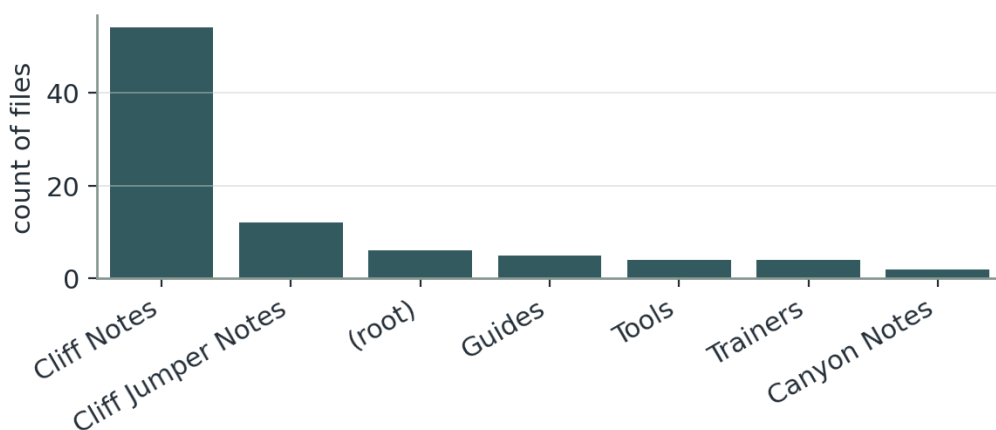
Ignore everything else and look at one column's shape. Bar charts for categories, histograms for measurements, boxplots for the compressed view.

Module 8 named the starting point: a **marginal** distribution looks at one variable and ignores all the others. Marginal plots come first; relationships come after. The chooser is mechanical: categorical → bar chart of counts; continuous → histogram. In seaborn, the figure-level function for categorical plots is `catplot`, and `kind="count"` does the counting for you (it is `value_counts` drawn):

```
import seaborn as sns

sns.catplot(data=df, x="section", kind="count")
```

Marginal distribution of section (one bar per value, height = rows)

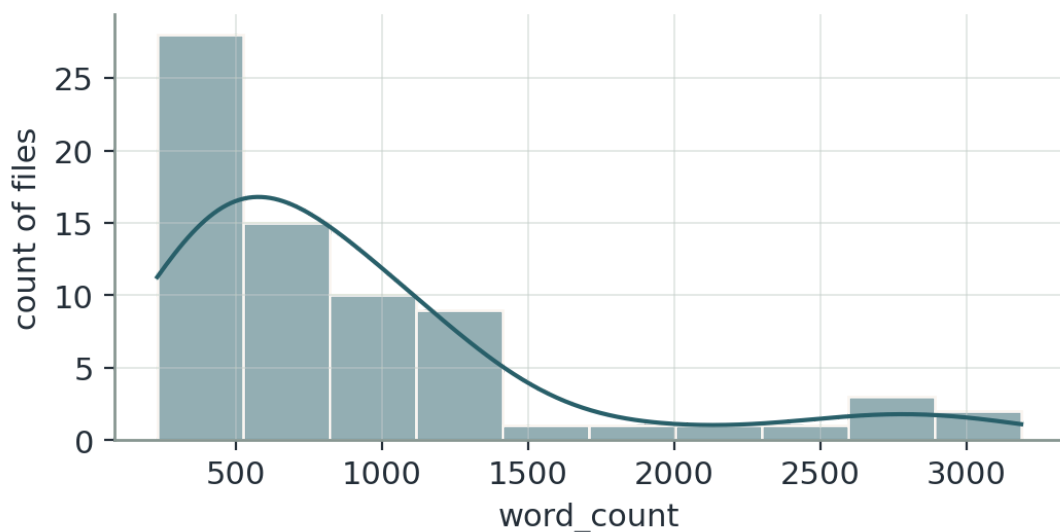


F1. The marginal distribution of `section`. One glance: *Cliff Notes* dominates with 54 of 87 files; *Canyon Notes* is the rare bird at 2. Exactly the numbers `value_counts()` printed, now visible.

For a continuous column, the histogram breaks the range into intervals called **bins** and counts rows per bin. The figure-level function is `displot`, and `kde=True` overlays the kernel density estimate, the smoothed cousin whose y-axis is density (concentration), not counts:

```
sns.displot(data=df, x="word_count", kde=True)
```

word_count: histogram with KDE overlay (71 text files)

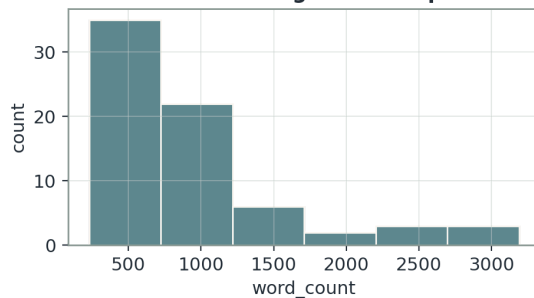


F2. `word_count` for the 71 text files. Most notes sit between roughly 300 and 1100 words; a long right tail of heavyweights reaches 3188. The KDE traces the same shape smoothly.

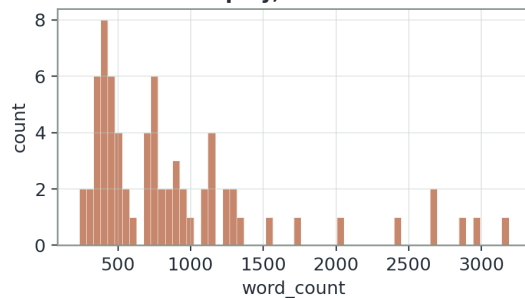
⚠ TRAP

Bins are a dial, and more is not better. Module 8's guidance: use a small number of bins for the general shape; a large dataset can support more; too many gives dramatic little spikes that carry little information.

bins=6: the general shape

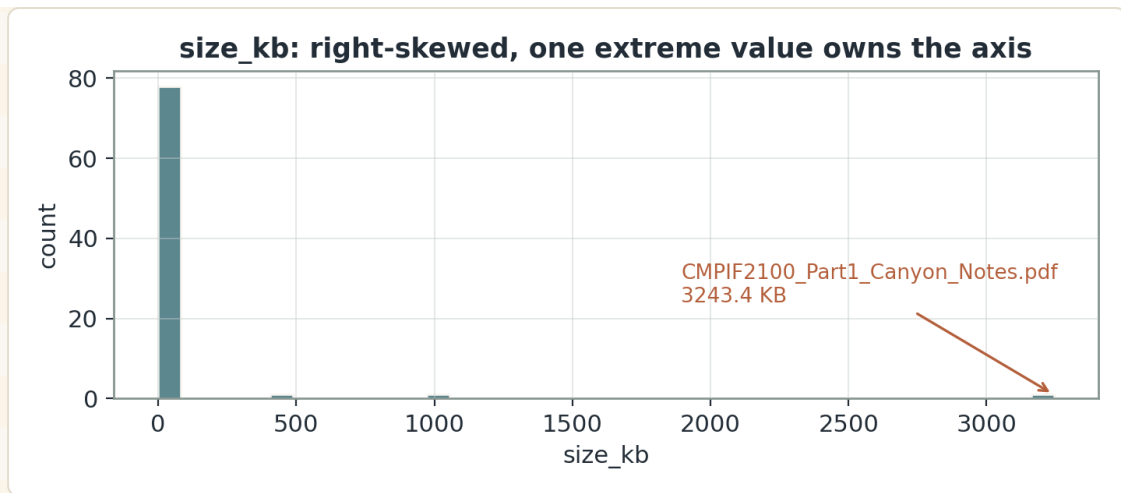


bins=60: spiky, little information



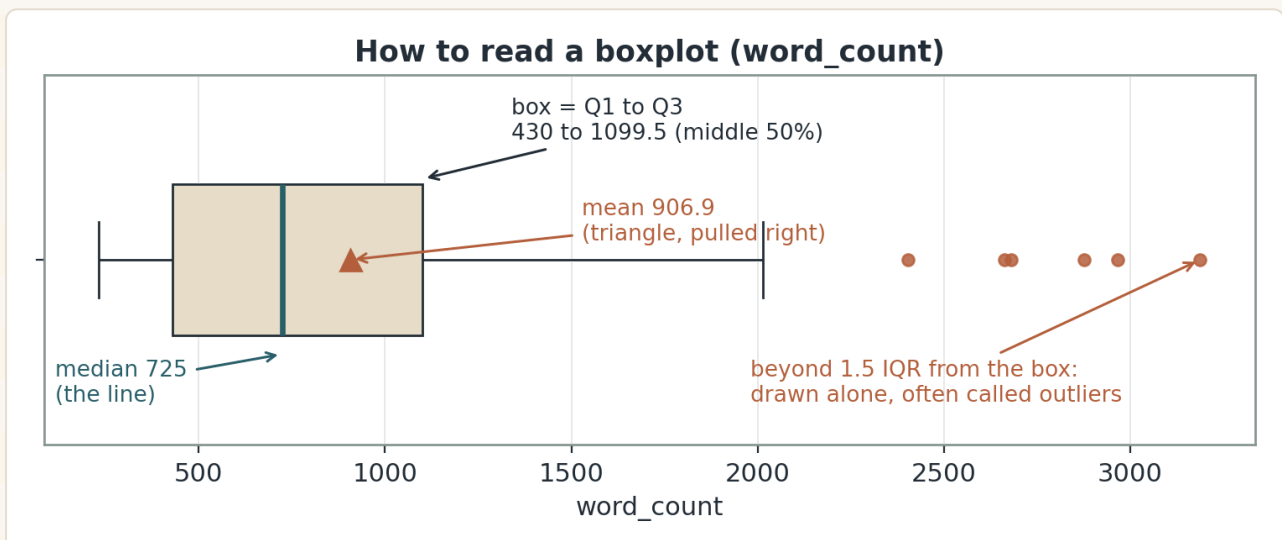
F3. Same 71 values, two dials: `bins=6` shows the shape, `bins=60` shows noise. When in doubt, go coarser.

Now the column `head()` warned us about. `size_kb` is the textbook skewed distribution: most values small, one enormous:



F4. `size_kb` : right-skewed with a vengeance. The 3243.4 KB Part 1 Canyon Notes PDF owns the axis; everything else huddles near zero; the 6 sizeless submodules sit it out entirely. This is why its mean (64.1) and median (5.1) disagree by more than a factor of twelve.

The third marginal view is the boxplot: the distribution compressed to five-ish landmarks. Module 8 taught its anatomy on penguins; here it is on our own words:



F5. How to read a boxplot, on `word_count` . The line is the median (725: half above, half below). The box spans quartile 1 to quartile 3 (430 to 1099.5), the middle 50%, whose length is the IQR (interquartile range). Whiskers reach the most extreme points within 1.5 IQR of the box edges; beyond that, points are drawn alone and often called outliers. The triangle is the mean (`showmeans=True`), pulled right of the median by the tail, just as Part IV predicted.

KEY IDEA

The plot chooser so far: categorical → count bar chart. Continuous → histogram (KDE for a smooth read). Want the compressed comparison view → boxplot. And always: marginal first. You cannot interpret a relationship between two variables whose individual shapes you have never seen.

Relationships: two variables at once

Three pairings, three plot families. This is where the fun begins.

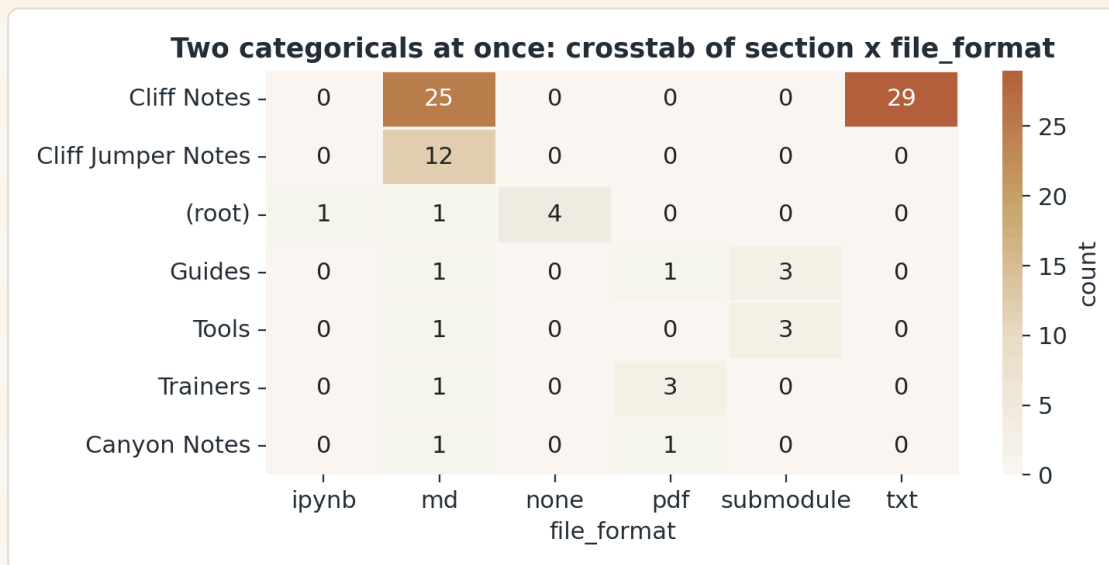
With marginals understood, ask how columns move together. The pairing of types picks the plot:

Pairing	Plot family	Seaborn move
categorical × categorical	grouped bars, or a heatmap of the crosstab	<code>countplot(..., hue=)</code> / <code>heatmap(pd.crosstab(...))</code>
categorical × continuous	boxplots per category (means: point plots)	<code>catplot(kind="box")</code>
continuous × continuous	scatter, fit line, correlation, pair plot	<code>relplot</code> / <code>lmpplot</code> / <code>.corr()</code> + <code>heatmap</code> / <code>pairplot</code>

The relationship chooser. Same driver as always: the data types involved.

Categorical × categorical

```
import pandas as pd
ct = pd.crosstab(df["section"], df["file_format"])
sns.heatmap(ct, annot=True, fmt="d")
```

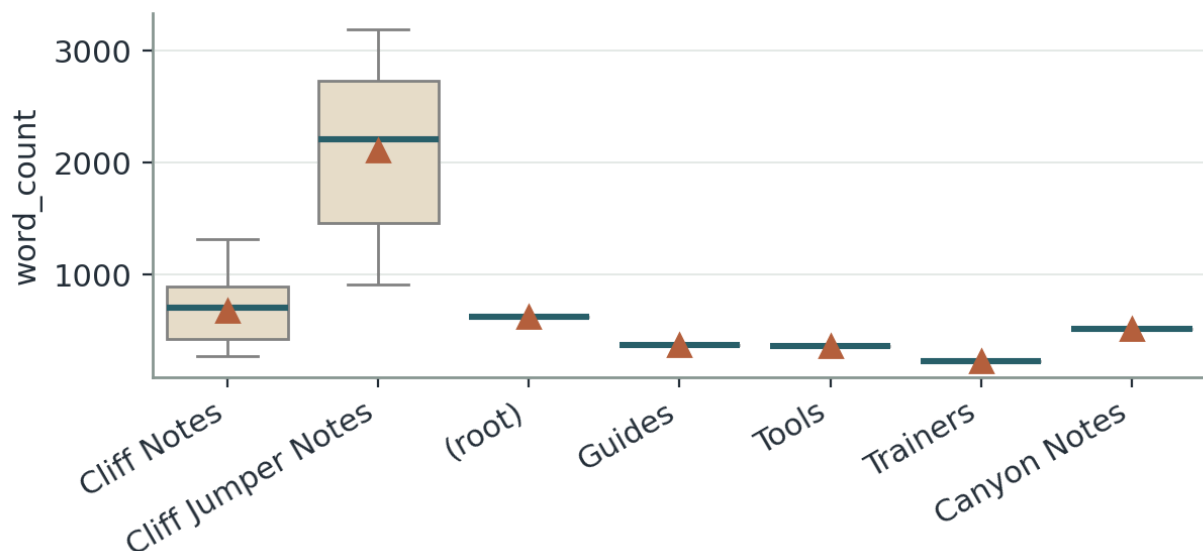


F6. Counts across two categoricals. Stories jump out: *Cliff Notes* is the only section mixing *md* and *txt* (25 *md* + 29 *txt*); every *Cliff Jumper* note is *md*; the *Trainers* hold 3 of the 5 *PDFs*; all 6 submodules live under *Guides* and *Tools*; the lone notebook sits at *root*.

Categorical × continuous

```
sns.catplot(data=df, x="section", y="word_count", kind="box", showmeans=True)
```

word_count by section: medians and spreads at a glance

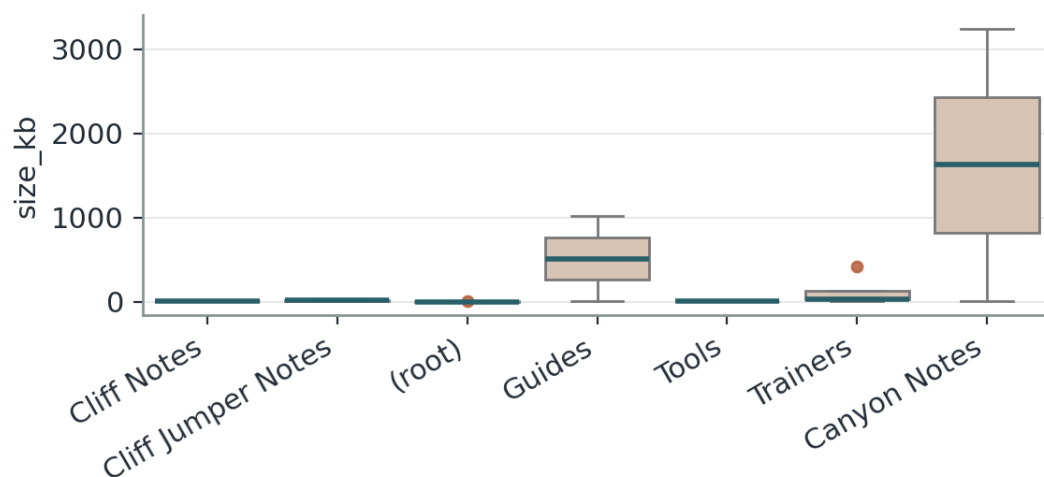


F7. One boxplot per category, medians and spreads at a glance. Cliff Jumper notes are in their own weight class (mean 2114 words, roughly triple the Cliff Notes' 684): the deeper tier really is deeper, and now you have the picture to prove it.

⚠ TRAP

Boxplots are at their best for roughly symmetric distributions. Feed them heavy skew and you get clutter that hides the shape. Module 8's advice when a boxplot looks like the one below: switch to a faceted histogram or a conditional KDE instead. (Our F4 histogram already did that job for `size_kb`.)

The trap: heavy skew makes boxplots unreadable (size_kb)



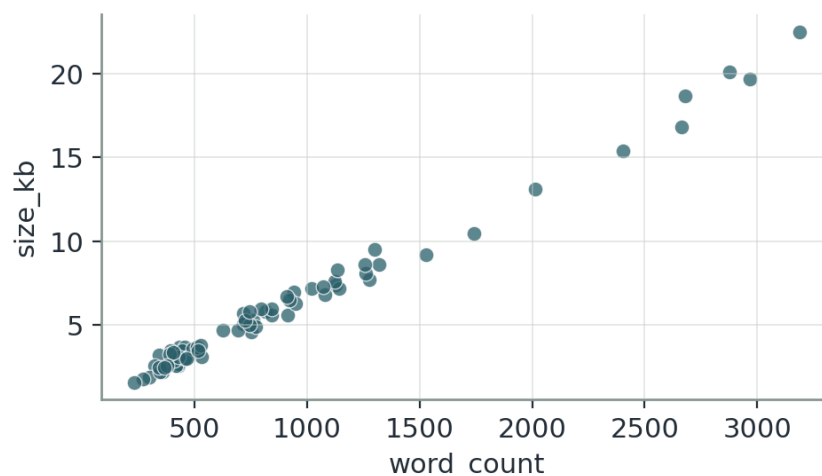
F8. The trap, demonstrated: `size_kb` by section. Most boxes squashed to slivers, one 3.2 MB file stretching the axis. The plot is not wrong; it is the wrong plot.

Continuous × continuous

The fundamental plot is the scatter (Module 9): every dot is a data point, meaning one row of the table, positioned by its two values. `relplot` draws it (scatter is the default kind):

```
sns.relplot(data=df, x="word_count", y="size_kb")
```

word_count vs size_kb: $r = 0.99$, two measurements of one thing



F9. `word_count` vs `size_kb`: a near-perfect line, $r = 0.99$. Almost mechanical, and that is the lesson: more words directly make a bigger file, so these are close to two measurements of one underlying thing. A huge correlation is often a discovery about your bookkeeping, not about the world.

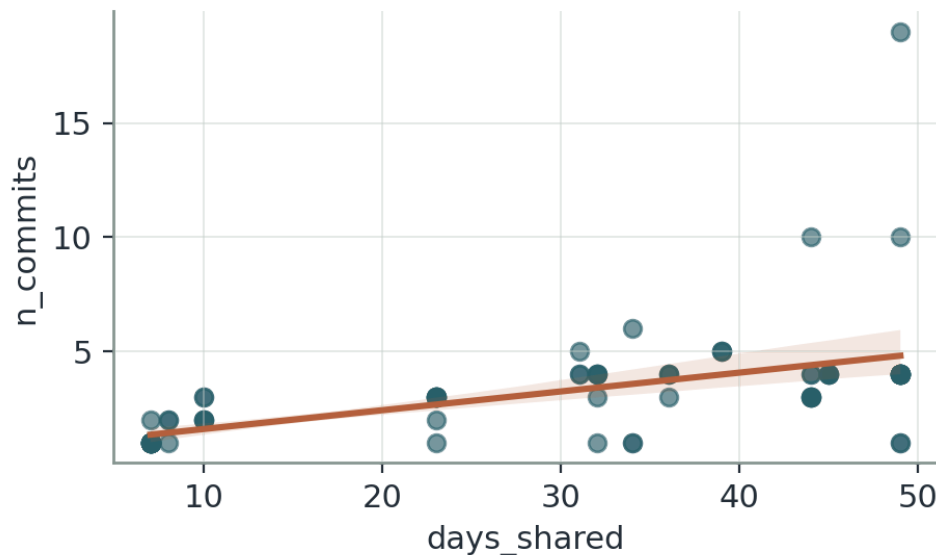
Module 10 turned "how related does it look" into a number: the **correlation coefficient**, a single summary statistic quantifying the linear relationship between two continuous variables. It runs

from -1 to +1: positive means they rise together, negative means one falls as the other rises.

`lmpplot` draws the best-fit line so you can see the sign as a slope:

```
sns.lmpplot(data=df, x="days_shared", y="n_commits")
```

`n_commits` vs `days_shared`: $r = 0.56$, the line slopes up



F10. `n_commits` vs `days_shared`, $r = 0.56$. The line slopes up: files that have been in the Share longer have collected more revisions. Sensible, and finally a real signal about our outcome.

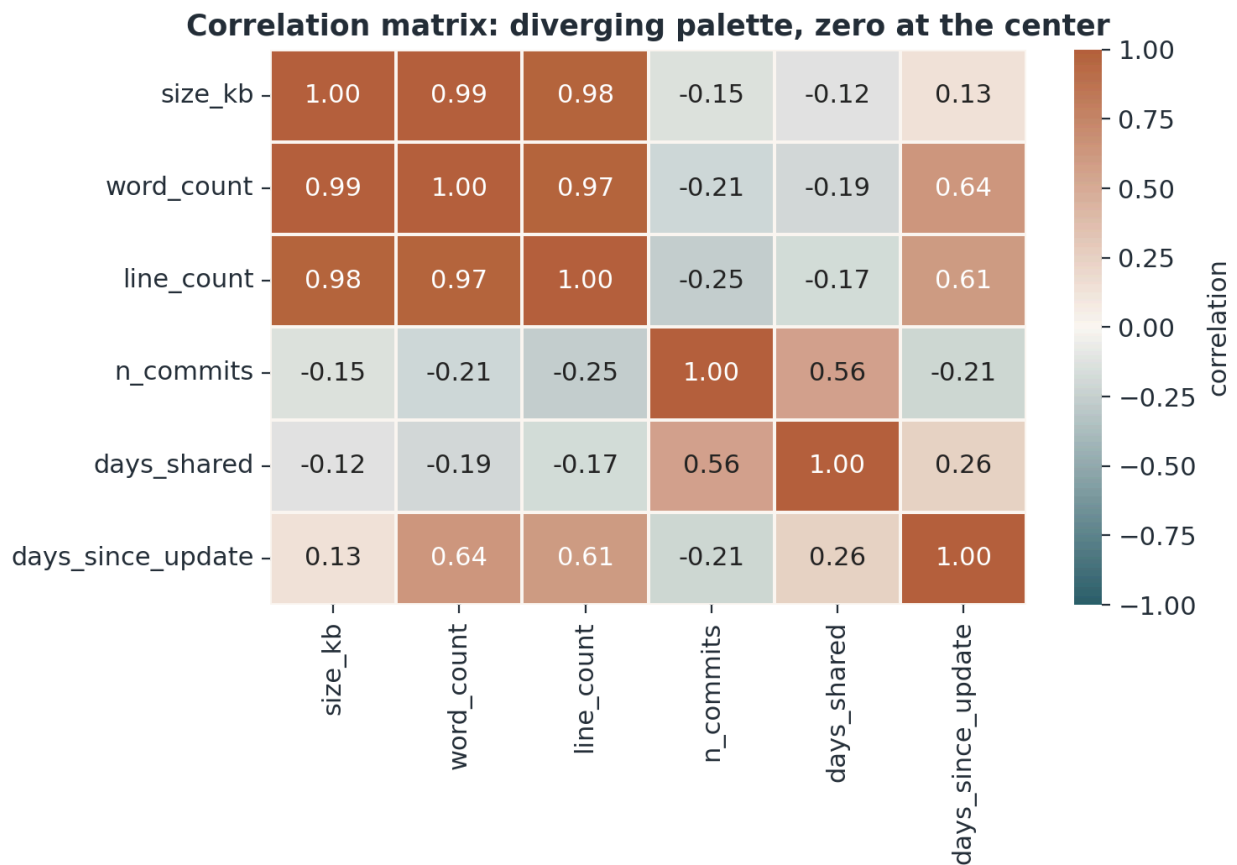
COURSE'S-WAY NOTE

Module 10, nearly verbatim: you do not have to squint. If it does not look very correlated, it is not very correlated. Say so plainly when asked.

With six continuous columns, pairwise scatters pile up fast. Two overview tools, both from Module 10. `.corr()` computes the whole correlation matrix (1s on the diagonal, every variable agrees with itself perfectly), and a heatmap makes it readable. Because correlations live on a scale with a meaningful zero, the heatmap wants `vmin=-1`, `vmax=1`, `center=0` and a diverging palette: two hues, one per sign, intensity showing distance from zero. `annot=True` prints the numbers on the cells:

```
con = ["size_kb", "word_count", "line_count", "n_commits", "days_shared", "days_since_update"]
corr = df[con].corr()

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))          # heatmap is axes-level: size it via matplotlib
sns.heatmap(corr, annot=True, fmt=".2f", vmin=-1, vmax=1, center=0)
```

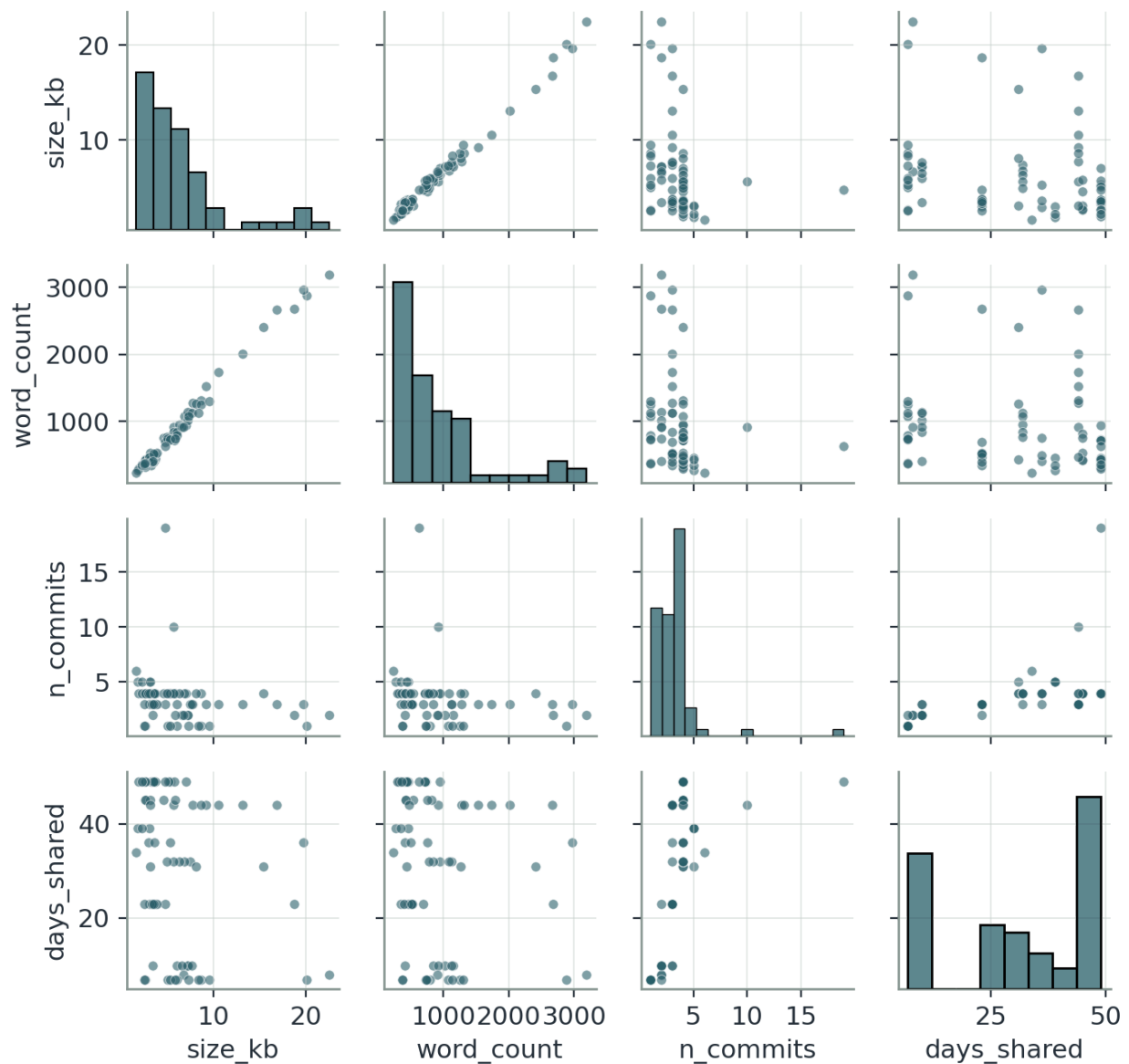


F11. The correlation matrix. The size/words/lines block glows near +1 (one underlying thing, three rulers). Our outcome's row is quieter: `n_commits` correlates 0.56 with `days_shared` and mildly negatively with the size family (-0.15 to -0.25): bigger notes are revised slightly less, older files revised more.

The visual twin is the pair plot: scatters for every pair at once, distributions on the diagonal:

```
sns.pairplot(df[["size_kb", "word_count", "n_commits", "days_shared"]])
```

Pair plot: every continuous pair at once (71 rows with word_count)



F12. Four columns, every pairing. The word/size wall of points confirms F9; the commit column shows its gentle structure against the rest. Pair plot for the overview, correlation matrix for the summary: Module 10 taught them as a pair.

KEY IDEA

Correlation measures how tightly points hug a straight line, not how steep the line is, and it only speaks about linear relationships. The number is a summary of a picture you should also look at.

Grouped relationships: does the trend survive the split?

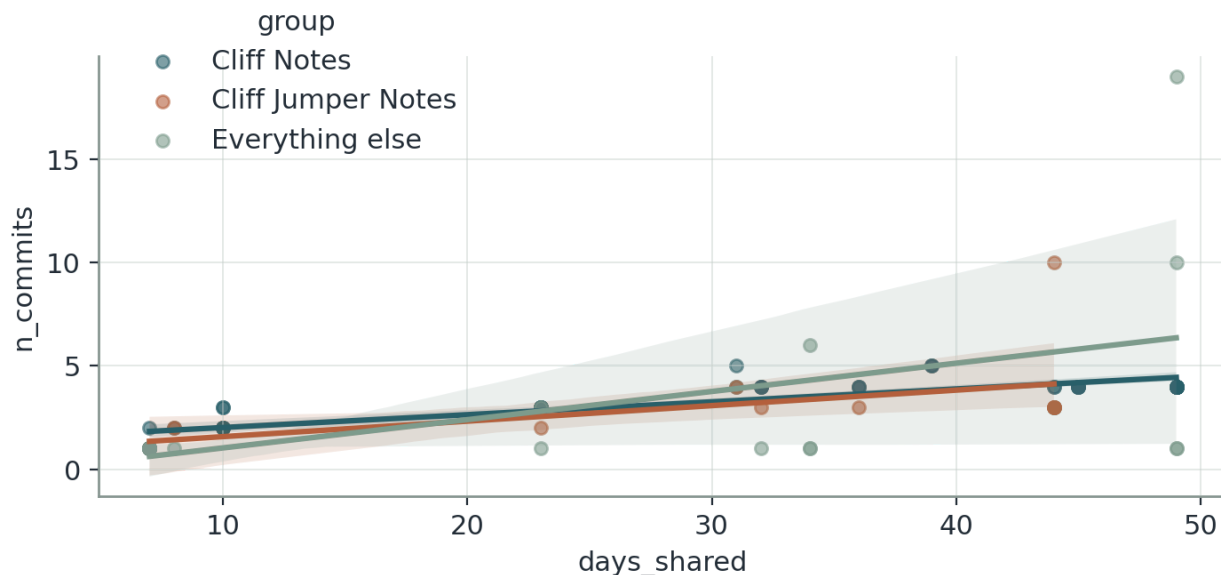
Take any relationship from Part VI and ask it again, once per group. Sometimes the answer changes.

Module 9 showed the move: a scatter plot can carry a third, categorical variable as `hue` (color), or as column and row facets. And it showed why the move matters with the penguins: bill depth vs flipper length trends one way when all species are pooled, and the *opposite* way inside each species. A pooled trend can change, weaken, or flat-out reverse once you group by a category. If your data has natural groups, the pooled picture is a hypothesis, not a conclusion.

```
import numpy as np

# three buckets, in a new column
df["group"] = np.where(df["section"] == "Cliff Notes", "Cliff Notes",
                      np.where(df["section"] == "Cliff Jumper Notes",
                              "Cliff Jumper Notes", "Everything else"))
sns.lmplot(data=df, x="days_shared", y="n_commits", hue="group")
```

Grouping sharpens it: $r = 0.56$ pooled, 0.86 inside Cliff Notes



F13. Our F10 relationship, split three ways. Pooled r was 0.56; inside Cliff Notes alone it is 0.86; inside Cliff Jumper notes, 0.47. Read the teal line carefully: it is shallow but tight. The dots hug it, and that hug is what $r = 0.86$ measures. The green "everything else" line is steeper but scattered (that group contains the 19-commit README). Slope is not strength; tightness is strength. Module 10 did say a near-flat line means little correlation: that rule pictures a plot scaled to its own data, and these shared axes flatten the teal line.

The penguins flipped; our notes sharpened. Both are the same lesson from opposite ends: the grouped view is where relationships tell the truth. For us it says the "older files collect more

commits" story is strongest precisely in the steady, wave-by-wave world of Cliff Notes, and muddier in the mixed everything-else group.

→ **THROUGH-LINE**

Split-apply-combine, again. `hue=` is a `groupby` you can see: split by the category, apply "fit a line," combine on one canvas. Facets are the same split drawn as separate panels. Module 7's engine did not stay in Module 7.

⚠ **TRAP**

Facets multiply fast. Module 9's demo stacked column facets, row facets, and hue until combinations ran out of data points. Group by what structures your data, not by everything you can.

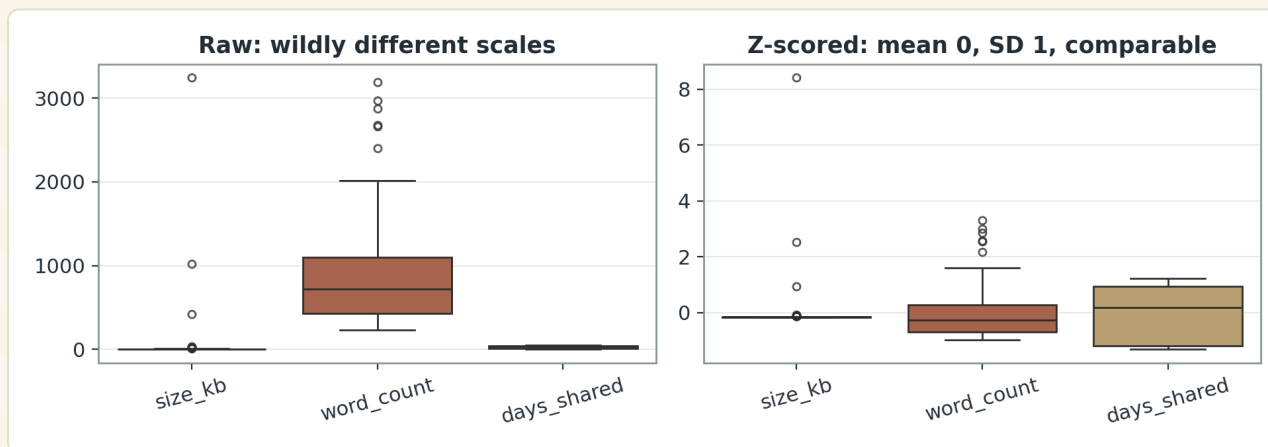
From exploring to predicting

What all of this was for: the outcome variable, and the doorway into modeling.

Return to the pipeline. Step 5 relates other attributes to the one you care about, the outcome. Our EDA kept `n_commits` in view and can now hand step 5 a briefing: revision count rises with age in the Share (0.56 pooled, 0.86 among Cliff Notes), drifts slightly against size (bigger notes, marginally fewer revisions), and its extremes are infrastructure files rather than study notes. That is what exploration owes modeling: not answers, but informed suspicions about which attributes matter.

One more Module 10 idea completes the doorway. Our continuous columns live on wildly different ranges (`size_kb` reaches 3243, `days_shared` tops out at 49). Techniques that measure distances or compare effects across variables need them on similar scales. **Z-standardization** transforms a column to mean 0 and standard deviation 1 by converting each value to its z-score: subtract the column's mean, divide by its sample standard deviation.

```
z = (df["word_count"] - df["word_count"].mean()) / df["word_count"].std()
z.mean(), z.std()    # -> -0.0, 1.0
```



F14. Left: the raw columns on one shared axis. The scales differ so much that only `word_count`'s box is readable: `size_kb` lies flat beneath its 3243.4 outlier and `days_shared` (7 to 49) barely registers, which is Module 10's own way of showing the problem. Right: the same three columns z-scored, mean 0 and SD 1 each, finally comparable.

Executed check: z-scored `word_count` comes back mean `-0.0`, std `1.0`.

KEY IDEA

Standardization does not change a distribution's shape, only its ruler. It is pre-processing: the quiet preparation work (Module 10 files it with cleaning, the "80%" world) that makes the modeling techniques ahead behave.

A closing habit from the triage days: `df.select_dtypes("number")` gathers every numeric column in one move (Module 8 used it to loop histograms). On our table it returns seven columns, and

one of them is `module`. The tool gathers numerics; the analyst re-sorts them. Which is Part III's lesson wearing step 5's coat, and a fitting last trap to leave armed.

THE WEAVE, IN ONE PARAGRAPH

Module 1 drew the pipeline and handed us the table. Module 6 loaded it and let us look. Modules 1, 6, 7, and 8 taught the triage: types, uniques, missingness, labels. Modules 4 and 7 summarized it: counts, moments, $n-1$, and the groupby engine. Module 8 drew one variable at a time; Modules 8, 9, and 10 drew them in pairs; Modules 7 and 9 split the pairs by groups and made the trends tell the truth. Module 10 standardized the rulers and opened the door to step 5. That is an EDA: not eight chores, one arc. And we ran the whole arc on the Note Share itself, 87 files you already know, so every claim in this document was checkable twice: once by code, once by memory.

Provenance and confidence

Written and directed by **Victor S. (DatJavaClass)** for the CMPIF Cohort Note Share. Built under the Double Diamond QC process: a truth diamond (author plus two independent adversarial auditors plus a meta-auditor) locked a verified fact spine; a clarity diamond enriched it and was audited the same way; a full-document release gate re-checked truth, clarity, and presentation before this PDF was rendered. Every number and figure was executed on python 3.13.9, numpy 2.3.5, pandas 2.3.3, matplotlib 3.10.6, seaborn 0.13.2 (the Anaconda stack these notes are built and verified on), against `noteshare_snapshot_2026-07-17.csv`, snapshotted from the public CMPIFCohortNoteShare repository on 2026-07-17.

Confidence: High across all eight parts. Concept claims trace to module lectures and notebooks; dataset claims were executed independently three times (author, auditor, meta-auditor). Per-file git history numbers (`n_commits` , day counts) carry Medium-High confidence: the snapshot's file census was verified against the live repository tree, while the per-file commit walks rest on the build's git queries.

The Double Diamond scorecard

Stage	Result
Diamond 1 (truth)	a 79-entry fact spine locked: every claim traced to module material or executed live; two adversarial audits plus a meta-audit; all fixes applied before the lock.
Diamond 2 (clarity)	enrichment built over the locked spine, then audited twice more: every figure and trace independently re-executed, and every spine entry re-walked for fidelity; a second meta-audit set the final fix list.
Figures	14 executed figures shipped (F1 to F14), one over the planned 10 to 13: F3 and F8 stayed as compact demos of the bins trap and the skew trap. Nothing was cut silently.
Release gate	before render: every number re-executed against the shipped CSV on the anchor stack, register and integrity scans re-run, every figure re-inspected against its caption.

This is a study aid, not a textbook, and not a substitute for doing your own analysis. Check anything important against the actual course material, especially before a quiz.

As always: you're awesome, stay awesome, and I wish everyone the best of grades!